

Practica 1

1) Características nuevas por década en lenguajes de programación:

1951 - 1955: Lenguajes tipo assembly:

- **Características:**
 - Código orientado a la máquina.
 - Bajo nivel de abstracción.
 - Dificultad de aprendizaje y uso.

1956 - 1960: FORTRAN, ALGOL 58, ALGOL 60, LISP:

- **Características nuevas:**
 - Notación matemática más legible.
 - Estructuras de control como bucles y condicionales.
 - Funciones y subrutinas.
 - Mayor facilidad de uso que el lenguaje ensamblador.

1961 - 1965: COBOL, ALGOL 60, SNOBOL, JOVIAL:

- **Características nuevas:**
 - Lenguajes orientados a problemas específicos (COBOL para negocios, JOVIAL para aplicaciones militares).
 - Manipulación de cadenas de caracteres.
 - Estructuras de datos más complejas.

1966 - 1970: APL, FORTRAN 66, BASIC, PL/I, SIMULA 67, ALGOL-W:

- **Características nuevas:**
 - Programación estructurada.
 - Tipos de datos más sofisticados.
 - Multitarea.
 - Simulación de sistemas.

1971 - 1975: Pascal, C, Scheme, Prolog:

- **Características nuevas:**
 - Programación modular.
 - Tipado estático.
 - Lenguajes de lógica para la inteligencia artificial.

1976 - 1980: Smalltalk, Ada, FORTRAN 77, ML:

- **Características nuevas:**
 - Programación orientada a objetos.
 - Concurrencia.
 - Lenguajes funcionales.

1981 - 1985: Smalltalk 80, Turbo Pascal, Postscript:

- **Características nuevas:**
 - Entornos de desarrollo integrados.

- Programación visual.
- Lenguajes de descripción de páginas.

1986 - 1990: FORTRAN 90, C++, SML:

- **Características nuevas:**
 - Programación genérica.
 - Plantillas de clases.
 - Lenguajes de scripting.

1991 - 1995: TCL, PERL, HTML:

- **Características nuevas:**
 - Lenguajes para el desarrollo web.
 - Interfaz gráfica de usuario.
 - Programación dinámica.

1996 - 2000: Java, Javascript, XML:

- **Características nuevas:**
 - Lenguajes multiplataforma.
 - Programación orientada a componentes.
 - Lenguajes de marcado para la web.

Lenguajes que incorporaron las características:

- **FORTRAN:** Lenguaje científico con foco en la eficiencia.
- **ALGOL:** Lenguaje de propósito general con enfoque en la elegancia y la legibilidad.
- **COBOL:** Lenguaje de negocios con foco en la facilidad de uso.
- **C:** Lenguaje de propósito general con foco en la eficiencia y la portabilidad.
- **Pascal:** Lenguaje de enseñanza con foco en la claridad y la estructuración.
- **Smalltalk:** Lenguaje de programación orientada a objetos con foco en la flexibilidad y la expresividad.
- **Java:** Lenguaje multiplataforma con foco en la seguridad y la robustez.

2) Historia del Lenguaje de Programación JavaScript

1995: JavaScript fue creado por Brendan Eich en Netscape Communications como un lenguaje de scripting del lado del cliente para el navegador web Netscape Navigator. Su nombre original era Mocha, luego LiveScript, y finalmente se estableció como JavaScript.

1997: JavaScript se convirtió en parte del estándar ECMAScript, lo que le dio una base sólida y una mayor adopción por parte de otros navegadores web.

2005: La llegada de Ajax popularizó el desarrollo web dinámico, lo que permitió a JavaScript realizar cambios en las páginas web sin necesidad de recargarlas.

2009: Node.js, una plataforma de desarrollo para ejecutar JavaScript en el servidor, amplió considerablemente las posibilidades de JavaScript, permitiendo su uso en aplicaciones web back-end y otras áreas.

2015: La introducción de ES6 (ECMAScript 2015) trajo consigo nuevas características a JavaScript, como clases, módulos, y promesas, mejorando su expresividad y modularidad.

Hoy en día: JavaScript es el lenguaje de programación más popular del mundo, utilizado para el desarrollo web front-end y back-end, aplicaciones móviles, juegos, Internet de las Cosas (IoT), y mucho más. Su popularidad se debe a su flexibilidad, versatilidad, y la gran cantidad de frameworks y bibliotecas disponibles.

3) Atributos de un Buen Lenguaje de Programación

Un buen lenguaje de programación debe tener una serie de características que lo hagan efectivo, eficiente y fácil de usar. Algunas de las más importantes son:

- **Ortogonalidad:** Los conceptos del lenguaje deben ser independientes y combinables entre sí sin restricciones innecesarias. Esto permite crear programas modulares y fáciles de entender.
- **Expresividad:** El lenguaje debe permitir expresar ideas y algoritmos de forma clara y concisa. Esto facilita la escritura y el mantenimiento del código.
- **Legibilidad:** El código escrito en el lenguaje debe ser fácil de leer y comprender, tanto para el propio programador como para otros. Esto ayuda a evitar errores y facilita la colaboración.
- **Simplicidad:** El lenguaje debe tener una sintaxis clara y concisa, sin reglas innecesarias o complejas. Esto facilita su aprendizaje y uso.
- **Eficiencia:** El lenguaje debe permitir la creación de programas eficientes que usen los recursos del sistema de forma óptima.
- **Portabilidad:** El código escrito en el lenguaje debe poder ejecutarse en diferentes plataformas sin necesidad de modificaciones.
- **Seguridad:** El lenguaje debe proporcionar mecanismos para proteger los datos y evitar vulnerabilidades.
- **Comunidad activa:** Un buen lenguaje de programación tiene una comunidad activa de usuarios y desarrolladores que pueden proporcionar ayuda y soporte.

Ejemplo: Python

Python es un lenguaje de programación que cumple con muchos de los atributos mencionados anteriormente. Es un lenguaje ortogonal, expresivo, legible, simple y eficiente. Además, es portable, seguro y tiene una comunidad activa de usuarios y desarrolladores.

4) JavaScript y Python: Expresiones, Organización y Atributos

Expresiones

JavaScript:

- **Operadores aritméticos:** +, -, *, /, %, etc.
- **Operadores relacionales:** <, >, <=, >=, ==, !=
- **Operadores lógicos:** &&, ||, !
- **Operadores de asignación:** =, +=, -=, *=, /=, etc.
- **Expresiones condicionales:** `if...else`
- **Bucles:** `for`, `while`, `do...while`
- **Funciones:** Declaración y llamada

Python:

- **Operadores aritméticos:** +, -, *, /, %, //, **
- **Operadores relacionales:** <, >, <=, >=, ==, !=, is, is not
- **Operadores lógicos:** and, or, not
- **Operadores de asignación:** =, +=, -=, *=, /=, etc.
- **Expresiones condicionales:** `if...elif...else`
- **Bucles:** `for`, `while`, `do...while`
- **Funciones:** Declaración y llamada

Organización del Programa

JavaScript:

- Funciones
- Objetos
- Módulos
- Clases (ES6)

Python:

- Funciones
- Clases
- Módulos
- Paquetes

Atributos

Atributo	JavaScript	Python	Justificación
Ortogonalidad	Media	Alta	JavaScript permite algunas combinaciones poco intuitivas.
Expresividad	Alta	Alta	Ambos lenguajes permiten expresar ideas complejas de forma concisa.
Legibilidad	Alta	Alta	La sintaxis de ambos lenguajes es similar al lenguaje natural.
Simplicidad	Alta	Alta	Ambos lenguajes tienen un conjunto de reglas relativamente simples.
Eficiencia	Media	Alta	Python generalmente es más eficiente que JavaScript.

Portabilidad	Alta	Alta	Ambos lenguajes son portables a diferentes plataformas.
Seguridad	Media	Alta	Python tiene características de seguridad más robustas que JavaScript.
Comunidad activa	Alta	Alta	Ambos lenguajes tienen comunidades activas de usuarios y desarrolladores.

11) Programas en Python: Paradigma y Características

Tipos de programas

Con Python se pueden escribir una gran variedad de programas, incluyendo:

- **Desarrollo web:** Back-end con frameworks como Django o Flask, front-end con frameworks como React o Angular, aplicaciones web con herramientas como Dash o Streamlit.
- **Análisis de datos:** Manipulación de datos con NumPy y Pandas, visualización de datos con Matplotlib y Seaborn, aprendizaje automático con scikit-learn y TensorFlow.
- **Automatización:** Creación de scripts para automatizar tareas repetitivas, interacción con APIs y sistemas operativos.
- **Ciencia e ingeniería:** Simulaciones numéricas, análisis de datos científicos, control de dispositivos.
- **Juegos:** Desarrollo de juegos simples o complejos con frameworks como PyGame o Godot.
- **Escritorio:** Creación de aplicaciones de escritorio con frameworks como Tkinter o wxPython.

Paradigma

Python es un lenguaje **multiparadigma**, lo que significa que admite diferentes estilos de programación:

- **Programación imperativa:** Se centra en la secuencia de instrucciones que modifican el estado del programa.
- **Programación orientada a objetos:** Se basa en la creación de objetos que encapsulan datos y comportamiento.
- **Programación funcional:** Se basa en la evaluación de funciones para producir resultados.
- **Programación lógica:** Se basa en la definición de reglas y relaciones entre hechos.

Características que determinan la pertenencia a un paradigma

Las características que determinan la pertenencia de Python a un paradigma incluyen:

- **Imperativa:** Soporte para variables, asignación, condicionales y bucles.
- **Orientada a objetos:** Soporte para clases, objetos, herencia y polimorfismo.
- **Funcional:** Soporte para funciones de primera clase, lambdas y expresiones lambda.
- **Lógica:** Soporte para reglas, hechos y consultas.

Python no es un lenguaje puramente de un solo paradigma, sino que permite combinar diferentes estilos de programación según las necesidades del proyecto.

En resumen:

- Python es un lenguaje versátil que se puede usar para escribir una amplia variedad de programas.
- Es un lenguaje multiparadigma que admite diferentes estilos de programación.
- La elección del paradigma a utilizar dependerá de las necesidades del proyecto y las preferencias del desarrollador.

12) Otras características importantes de Python:

Tipado de datos:

- Python es un lenguaje de **tipado dinámico**, lo que significa que el tipo de dato de una variable se determina en tiempo de ejecución.
- Los tipos de datos básicos incluyen: `int`, `float`, `str`, `bool`, `list`, `tuple`, `dict`, `set`.
- El tipado dinámico facilita la escritura de código, pero puede ser menos eficiente y propenso a errores en comparación con el tipado estático.

Organización de programas:

- Los programas en Python se organizan en **módulos**, que son archivos que contienen definiciones de funciones, clases y variables.
- Los módulos se pueden importar y usar en otros módulos, lo que facilita la reutilización de código.
- Python también admite la creación de **paquetes**, que son colecciones de módulos relacionados.

Otras características:

- **Interpretado:** Python no necesita ser compilado antes de ejecutarse, lo que facilita el desarrollo y la depuración.
- **Portable:** Python se puede ejecutar en una amplia variedad de plataformas, incluyendo Windows, macOS, Linux y Android.
- **Gran biblioteca estándar:** Python tiene una biblioteca estándar extensa que incluye módulos para una variedad de tareas comunes.
- **Comunidad activa:** Python tiene una comunidad activa de usuarios y desarrolladores que brindan soporte y recursos.

En resumen:

- Python es un lenguaje de programación versátil y fácil de aprender con una amplia gama de características.

- Su tipado dinámico, organización modular, interpretabilidad, portabilidad y gran biblioteca estándar lo convierten en una herramienta poderosa para una variedad de aplicaciones.

13 y 14)

Paradigma: JavaScript es un lenguaje **multiparadigma**, lo que significa que puede usarse para implementar diferentes estilos de programación:

- **Programación imperativa:** Se centra en la secuencia de instrucciones que modifican el estado del programa.
- **Programación orientada a objetos:** Se basa en la creación de objetos que encapsulan datos y comportamiento.
- **Programación funcional:** Se basa en la evaluación de funciones para producir resultados.

Tipo de lenguaje: JavaScript es un lenguaje de **programación de alto nivel**, lo que significa que está más cerca del lenguaje humano que del lenguaje máquina. Esto lo hace más fácil de aprender y usar que los lenguajes de bajo nivel.

Tipado de datos:

- JavaScript es un lenguaje de **tipado dinámico**, lo que significa que el tipo de dato de una variable se determina en tiempo de ejecución.
- Los tipos de datos básicos incluyen: `number`, `string`, `boolean`, `null`, `undefined`, `object`, `symbol`.
- El tipado dinámico facilita la escritura de código, pero puede ser menos eficiente y propenso a errores en comparación con el tipado estático.

Excepciones:

- JavaScript permite el manejo de excepciones para controlar errores en tiempo de ejecución.
- Se pueden usar las palabras clave `try`, `catch` y `finally` para definir bloques de código que se ejecutan en caso de error, éxito o finalización de una operación.

Variables:

- Las variables en JavaScript se declaran con la palabra clave `var`, `let` o `const`.
- `var` es la forma tradicional de declarar variables, pero tiene alcance de función.
- `let` y `const` son nuevas formas de declarar variables con alcance de bloque.
- `const` se usa para declarar variables constantes que no se pueden modificar.

Otras características:

- **Interpretado:** JavaScript no necesita ser compilado antes de ejecutarse, lo que facilita el desarrollo y la depuración.
- **Orientado a eventos:** JavaScript se basa en eventos para interactuar con el usuario y el entorno.
- **Gran biblioteca estándar:** JavaScript tiene una biblioteca estándar extensa que incluye módulos para una variedad de tareas comunes.
- **Comunidad activa:** JavaScript tiene una comunidad activa de usuarios y desarrolladores que brindan soporte y recursos.

En resumen:

- JavaScript es un lenguaje de programación versátil y fácil de aprender con una amplia gama de características.

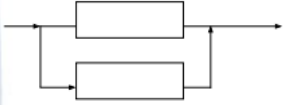
- Su tipado dinámico, manejo de excepciones, variables, interpretabilidad, orientación a eventos y gran biblioteca estándar lo convierten en una herramienta poderosa para el desarrollo web front-end y otras aplicaciones.

Práctica 2 - Sintaxis

Objetivo: conocer cómo se define léxicamente un lenguaje de programación y cuales son las herramientas necesarias para hacerlo.

Ejercicio 1

Complete el siguiente cuadro:

Meta símbolos utilizados por		Símbolo utilizado en Diagramas sintácticos	Significado
BNF	EBNF		
palabra terminal	palabra terminal		Definición de un elemento terminal
palabra no terminal	palabra no terminal		Definición de un elemento no terminal
::=	::=	diagrama con rectángulos, óvalos y flechas	Definición de una producción
	()		Selección (de una alternativa)
< p > < p1 >	{ }		Repetición
	*		Repetición de 0 o más veces
	+		Repetición de 1 o más veces
	[]		Opcional

Nota: p y p1 son producciones simbólicas

Ejercicio 2

¿Cuál es la importancia de la sintaxis para un lenguaje? ¿Cuáles son sus elementos?

Es importante porque establece las reglas y la estructura que deben seguir los programas. Determina cómo deben organizarse y escribirse las instrucciones y expresiones para que el programa sea comprensible para el compilador o intérprete.

- Identificadores
- Operadores
- Palabras clave
- Palabras reservadas
- Comentarios
- Literales (10, "hola", true)
- Delimitadores ({}, (), ., ,, ;, :, ""')

Ejercicio 3

Explique a qué se denomina regla lexicográfica y regla sintáctica.

Reglas lexicográficas: conjuntos de reglas para formar las "word" (palabras clave), a partir de caracteres.

- Símbolo distinto: En C !=, en Pascal <>

Reglas sintácticas: conjuntos de reglas que definen cómo formar las "expresiones" y "sentencias".

- If: En C no lleva then, en Pascal sí

Ejercicio 4

¿En la definición de un lenguaje, a qué se llama palabra reservada? ¿A qué son equivalentes en la definición de una gramática? De un ejemplo de palabra reservada en el lenguaje que más conoce. (Ada,C,Ruby,Python,...)

Palabra reservada: palabra clave que no puede ser usada por el programador como identificador de otra identidad.

En la definición de una gramática son equivalentes a los símbolos no terminales.

- Pascal: and, array, begin, const, div, do, downto, else, if, in, mod, not, procedure, record, then, to, var, while

Ejercicio 5

Dada la siguiente gramática escrita en BNF:

$G = (N, T, S, P)$

$N = \{ \langle \text{numero_entero} \rangle, \langle \text{digito} \rangle \}$

$T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$S = \langle \text{numero_entero} \rangle$

$P = \{$

$\langle \text{numero_entero} \rangle ::= \langle \text{digito} \rangle \langle \text{numero_entero} \rangle \mid \langle \text{numero_entero} \rangle \langle \text{digito} \rangle \mid \langle \text{digito} \rangle$

$\langle \text{digito} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$\}$

a- Identifique las componentes de la misma

G = 4-tupla de conjunto de reglas finitas que define un conjunto infinito de posibles sentencias válidas en el lenguaje.

N = conjunto de símbolos no terminales.

T = conjunto de símbolos terminales.

S = símbolo distinguido de la gramática que pertenece a N .

P = conjunto de producciones.

b- Indique porqué es ambigua y corríjala

Es ambigua porque la expresión " $\langle \text{digito} \rangle \langle \text{numero_entero} \rangle \mid \langle \text{numero_entero} \rangle \langle \text{digito} \rangle$ " indica que el árbol de derivación podría recorrer con recursión por izquierda o por derecha, lo que va en contra de las reglas. Debería borrarse una de las opciones para quitar la ambigüedad.

$\langle \text{numero_entero} \rangle ::= \langle \text{digito} \rangle \langle \text{numero_entero} \rangle \mid \langle \text{digito} \rangle$

Ejercicio 6

Defina en BNF (Gramática de contexto libre desarrollada por Backus- Naur) la gramática para la definición de una palabra cualquiera.

Ejercicio 7

Defina en EBNF la gramática para la definición de números reales. Inténtelo desarrollar para BNF y explique las diferencias con la utilización de la gramática EBNF.

Ejercicio 8

Utilizando la gramática que desarrolló en los puntos 6 y 7, escriba el árbol sintáctico de:

a. Conceptos

b. Programación

c. 1255869

d. 854,26

e. Conceptos de lenguajes

Ejercicio 9

Defina utilizando diagramas sintácticos la gramática para la definición de un identificador de un lenguaje de programación. Tenga presente como regla que un identificador no puede comenzar con números.

Ejercicio 10

- Defina con EBNF la gramática para una expresión numérica, dónde intervienen variables y números. Considerar los operadores +, -, * y / sin orden de prioridad. No considerar el uso de paréntesis.
- A la gramática definida en el ejercicio anterior agregarle prioridad de operadores.
- Describa con sus palabras los pasos y decisiones que tomó para agregarle prioridad de operadores al ejercicio anterior.

Ejercicio 11

La siguiente gramática intenta describir sintácticamente la sentencia for de ADA, indique cuál/cuáles son los errores justificando la respuesta.

N = { <sentencia_for>, <bloque>, <variable>, <letra>, <cadena>, <digito>, <otro>, <operacion>, <llamada_a_funcion>, <numero>, <sentencia> }

P = {

<sentencia_for> ::= for (i= IN 1..10) loop <bloque> end loop;

<variable> ::= <letra> | <cadena>

<cadena> ::= { (<letra> | <digito> | <otro>) }+

<letra> ::= (a | .. | z | A | .. | Z)

<digito> ::= (1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0)

<bloque> ::= <sentencia> | <sentencia> <bloque> | <bloque> <sentencia> ;

<sentencia> ::= <sentencia_asignacion> | <llamada_a_funcion> | <sentencia_if> |

<sentencia_for> | <sentencia_while> | <sentencia_switch> }

- Faltan definir G, (elementos terminales) y S (símbolo distinguido) en la 4-tupla.
- En N faltan agregar los elementos <sentencia_asignación>, <sentencia_if>, <sentencia_while>, <sentencia_switch>
- Faltan definir los elementos anteriores.
- El rango escrito en <sentencia_for> debería ser un elemento no terminal.
- Se mezclan las notaciones BNF (<bloque>, <variable>, <sentencia_for>) y EBNF (<cadena>, <letra>, <digito>)
- El elemento <variable> es innecesario, ya que una <cadena> podría componerse por una sola <letra>.
- El componente <bloque> hace que la sintaxis sea ambigua, debido a que tiene definida recursión por izquierda y por derecha.

Ejercicio 12

Realice en EBNF la gramática para la definición un tag div en html 5. (Puede ayudarse con el siguiente enlace (<https://developer.mozilla.org/es/docs/Web/HTML/Elemento/div>))

G = (N, T, S, P)

N = { <div>, <atributos>, <atributo>, <etiqueta>, <etiquetas>, <bloque>, <id>, <string>, <char>, <special_char> }


```

T = {}
S = <div>
P = {
<div> ::= "<div " <atributos> ">" <bloque> "</div>"
<atributos> ::= { <atributo> }*
<atributo> ::= <id> "=" <string>
<etiquetas> ::= { <etiqueta> }*
<etiqueta> ::= "<" <id> <atributos> ">" ( <bloque> "</" <id> ">" | ">" )
<bloque> ::= ( <etiquetas> | <div> | <string> )
<id> ::= { <char> }+
<string> ::= { ( <char> | <special_char> ) }+
<char> ::= ( "a" | ... | "z" | "A" | ... | "Z" )
<special_char> ::= ( "-" | "_" | "/" | "+" | "*" | "." | "," | "\"" | "'" )
<operador> ::= ( "!=" )

```

Ejercicio 13

Defina en EBNF una gramática para la construcción de números primos. ¿Qué debería agregar a la gramática para completar el ejercicio?

```

G = (N, T, S, P)
N = { <numPrimo>, }
T = {}
S = <numPrimo>
P = {
}

```

Ejercicio 14

Sobre un lenguaje de su preferencia escriba en EBNF la gramática para la definición de funciones o métodos o procedimientos (considere los parámetros en caso de ser necesario)

```

G = (N, T, S, P) - Pascal
N = { <metodo>, <procedure>, <function>, <parametros>, <parametro>, <tipo>, <var>, <body>, <sentence>, <tipo>, <string>, <char>, <special_char>, <nums> }
T = { "a", ..., "z", "A", ..., "Z", 0, 1, ..., 8, 9, " ", ",", ".", ":", ";", "=", "+", "*" }
S = <metodo>
P = {
<metodo> ::= ( <function> | <procedure> )
<function> ::= "function " <string> "(" <parametros> ")": <tipo> ",",
<procedure> ::= "procedure " <string> "(" { <parametro> }* ")",
<parametros> ::= [ ( <parametro> | { <parametro> }+ ";" <parametro> ) ]
<parametro> ::= [ "var " ] <variable>
<tipo> ::= "type " { <string> "=" ( <string> | <tipo> | <variable> ) }+
<var> ::= "var " { <string> ":" ( <string> | <tipo> ) }+
<body> ::= { <sentence> }+
<sentence> ::= <string> ":",
<tipo> ::= ( "int" | "string" | "boolean" | "real" | <string> )
<string> ::= ( <char> | <special_char> ) { ( <char> | <special_char> | <nums> ) }+
<char> ::= ( "a" | ... | "z" | "A" | ... | "Z" )

```

```
<special_char> ::= ( " " | " , " | " . " | " : " | " ; " | " ' " | " = " | " + " | " * " )  
<nums> ( 0 | 1 | ... | 8 | 9 )  
}
```

Práctica 3 -Semántica

Objetivo: Interpretar el concepto de semántica de los lenguajes de programación.

Ejercicio 1: ¿Qué define la semántica?

Define el significado o interpretación que se le da a los símbolos, palabras, expresiones y sentencias en un lenguaje de programación sintácticamente válido.

Ejercicio 2

- a. ¿Qué significa compilar un programa?
 - b. Describa brevemente cada uno de los pasos necesarios para compilar un programa.
 - c. ¿En qué paso interviene la semántica y cual es su importancia dentro de la compilación?
-
- a) Compilar un programa es el proceso de traducir el código fuente de un programa escrito en un lenguaje de alto nivel a un formato que pueda ser entendido y ejecutable por la computadora. Esta traducción se puede realizar a lenguaje objeto (lenguaje máquina), el cual suele ser ejecutable, o a un lenguaje de nivel intermedio (lenguaje ensamblador).
 - b) **1. Análisis léxico (Etapa análisis-scanner):** el compilador escanea el código fuente y divide el texto en unidades léxicas llamadas tokens (palabras clave, identificadores, operadores, literales). Analiza cada uno de estos para ver si son válidos. Pone los identificadores en la tabla de símbolos.
2. Análisis sintáctico (análisis-parser): el compilador verifica que la estructura del código cumpla con las reglas gramaticales del lenguaje de programación mediante el análisis de la secuencia de tokens. Utiliza un árbol sintáctico para analizar los tokens.
3. Análisis semántico (análisis-semántica estática): el compilador verifica que el código cumpla con las reglas y restricciones semánticas del lenguaje de programación. Se centra en el significado y comportamiento del programa. Se realiza previo a ejecución y se centra en chequeo de tipos de datos, alcance de las variables, inicialización de variables, funciones utilizadas antes de su declaración, entre otros.
4. Optimización: es optativa. La puede realizar una herramienta independiente o el mismo compilador. Quien la realice puede eliminar código muerto, simplificar expresiones o reordenar instrucciones.
5. Generación del código: el compilador construye un programa ejecutable y genera el código necesario en un lenguaje intermedio en lugar de código máquina. Puede facilitar la optimización y la portabilidad del código.

Ejercicio 3

Con respecto al punto anterior ¿es lo mismo compilar un programa que interpretarlo? Justifique su respuesta mostrando las diferencias básicas, ventajas y desventajas de cada uno.

Compilación	Interpretación
Convierte el código fuente en un lenguaje de alto nivel a código máquina o código objeto antes de la ejecución.	Ejecuta el código fuente línea por línea, traduciendo y ejecutándolo en tiempo de ejecución.
Después de la compilación, el código puede ser ejecutado sin necesidad de volver a compilar.	Cada vez que se ejecuta, lo hace sin una compilación previa, permitiendo una ejecución más rápida del programa durante el desarrollo.
- Mayor velocidad de ejecución. - Detección de errores en compilación. - Independencia del código fuente después de la compilación. - Mayor protección de la propiedad del código, ya que este no se distribuye con el programa.	- Mayor facilidad y rapidez en el proceso de desarrollo y prueba. - Mayor portabilidad, ya que el mismo código puede ejecutarse en diferentes plataformas. - Mayor flexibilidad de depuración y modificación después de una ejecución.
- Mayor tiempo y recursos durante el desarrollo. - Menor portabilidad, ya que el código es específico por arquitectura y SO. - Menor flexibilidad de depuración y modificación después de la compilación.	- Menor velocidad de ejecución, ya que el código se ejecuta línea por línea. - Posible exposición del código fuente, ya que este se distribuye con el programa interpretado. - Se detectan errores en tiempo de ejecución, lo que hace más difícil depurar problemas complejos.

Ejercicio 4

Explique claramente la diferencia entre un error sintáctico y uno semántico. Ejemplifique cada caso.

Error sintáctico: ocurre cuando el código no sigue las reglas gramaticales del lenguaje de programación.

```
1 print "Hola mundo"
```

En este ejemplo, faltan los paréntesis que rodean el parámetro "Hola mundo", esto se debe a que en Python 3.X, print es una función y requiere paréntesis para indicar que se está llamando.

Error semántico: ocurre cuando el código sigue las reglas gramaticales del lenguaje de programación, pero viola las reglas semánticas del lenguaje.

```

1  a = "5"
2  resultado = a + 5

```

En este ejemplo, se está intentando sumar un string ("5") con un número entero (5). Aunque la expresión es sintácticamente válida, produce un error porque el operador "+" no está definido para concatenar un string con un número entero en Python.

Ejercicio 5

Sean los siguientes ejemplos de programas. Analice y diga qué tipo de error se produce (Semántico o Sintáctico) y en qué momento se detectan dichos errores (Compilación o Ejecución).

Aclaración: Los valores de la ayuda pueden ser mayores.

a) Pascal

```

Program P                                // Sintáctico, compilación
var 5: integer;                          // Sintáctico, compilación
var a:char;                              // Semántico, compilación
Begin
    for i:=5 to 10 do begin
        write(a);                        // Semántico, compilación
        a=a+1;                          // Sintáctico, compilación - Semántico, compilación
    end;
End.

```

End.

Ayuda: Sintáctico 2, Semántico 3

b) Java:

```

public String tabla(int numero, arrayList<Boolean> listado) {    // Semántico, compilación
    String result = null;
    for(i = 1; i < 11; i--) {                                     // Semántica,compilación-Lógico
        result += numero + "x" + i + "=" + (i*numero) + "\n"; // Semántico, ejecución
                                                                // Semántico, compilación
        listado.get(listado.size()-1)=(BOOLEAN) numero>i; // Sintáctico, compilación
    }
    return true;                                                // Semántico, compilación
}

```

Ayuda: Sintácticos 4, Semánticos 3, Lógico 1

c) C

```

#include <stdio.h>
int suma; /* Esta es una variable global */
int main() {
    int indice;
    encabezado;                                                  // Sintáctico, compilación
    for (indice = 1 ; indice <= 7 ; indice ++)
        cuadrado (indice);
    final(); Llama a la función final */                        // Sintáctico, compilación- Semántico, ejecución
}

```

```

        return 0;
    }
    cuadrado (numero)
    int numero;
    {
        int numero_cuadrado;
        numero_cuadrado == numero * numero;
        suma += numero_cuadrado;
        printf("El cuadrado de %d es %d\n", numero, numero_cuadrado);
    } // Sintáctico, semántico (return)
Ayuda: Sintácticos 2, Semánticos 6

```

d) Python

```

#!/usr/bin/python
print "\nDEFINICION DE NUMEROS PRIMOS"
r = 1
while r = True:
    N = input("\nDame el numero a analizar: ")
    i = 3
    fact = 0
    if (N mod 2 == 0) and (N != 2):
        print "\nEl numero %d NO es primo\n" % N
    else:
        while i <= (N^0.5):
            if (N % i) == 0:
                mensaje="\nEl numero ingresado NO es primo\n" % N
                msg = mensaje[4:6]
                print msg
                fact = 1
            i+=2
        if fact == 0:
            print "\nEl numero %d SI es primo\n" % N
        r = input("Consultar otro número? SI (1) o NO (0)--->> ")
Ayuda: Sintácticos 2, Semánticos 3

```

e) Ruby

def ej1

```
Puts 'Hola, ¿Cuál es tu nombre?'
nom = gets.chomp
puts 'Mi nombre es ', + nom
puts 'Mi sobrenombre es 'Juan"
puts 'Tengo 10 años'
meses = edad*12
dias = 'meses' *30
hs= 'dias * 24'
puts 'Eso es: meses + ' meses o ' + dias + ' días o ' + hs + ' horas'
puts 'vos cuántos años tenés'
edad2 = gets.chomp
edad = edad + edad2.to_i
puts 'entre ambos tenemos ' + edad + ' años'
puts '¿Sabes que hay ' + name.length.to_s + ' caracteres en tu nombre, ' + name +
'?'
```

end

Ayuda: Semánticos +4

Ejercicio 6

Dado el siguiente código escrito en pascal. Transcriba la misma funcionalidad de acuerdo al lenguaje que haya cursado en años anteriores. Defina brevemente la sintaxis (sin hacer la gramática) y semántica para la utilización de arreglos y estructuras de control del ejemplo.

Procedure ordenar_arreglo(var arreglo: arreglo_de_caracteres;cont:integer);

var

i:integer;

ordenado:boolean;

aux:char;

begin

repeat

ordenado:=true;

for i:=1 to cont-1 do

if ord(arreglo[i])>ord(arreglo[i+1]) then begin

aux:=arreglo[i];

arreglo[i]:=arreglo[i+1];

arreglo[i+1]:=aux; ordenado:=false

end;

until ordenado;

end;

Observación: Aquí sólo se debe definir la instrucción y qué es lo que hace cada una; detallando alguna particularidad del lenguaje respecto de ella. Por ejemplo el for de java necesita definir una variable entera, una condición y un incremento para dicha variable.

```

1  //Java
2  public char[] ordenar_arreglo(char[] arreglo, int cont) {
3      int i;
4      boolean ordenado;
5      char aux;
6
7      do {
8          ordenado = true;
9          for (i = 1; i < cont-1; i++) {
10             if ( (int)arreglo[i] > (int)arreglo[i+1] ) {
11                 aux = arreglo[i];
12                 arreglo[i] = arreglo[i+1];
13                 arreglo[i+1] = aux;
14                 ordenado = false;
15             }
16         }
17     } while (!ordenado);
18     return arreglo;
19 }

```

Ejercicio 7

Explique cuál es la semántica para las variables predefinidas en lenguaje Ruby `self` y `nil`. ¿Qué valor toman; cómo son usadas por el lenguaje?

self: se refiere al receptor actual del mensaje. Es como **this** en Java.

- Dentro del contexto global, se refiere al objeto principal de la aplicación (objeto especial llamado "main").
- Dentro de un método de una instancia, se refiere a la instancia de la clase en la que se define el método.
- Dentro de un método de clase, se refiere a la propia clase.

Se utiliza para distinguir entre métodos de instancia y métodos de clase, para acceder a variables y métodos de instancia, y para hacer referencia al propio objeto en el que se está trabajando.

nil: representa la ausencia de un valor. Es como **null** en Java. Es el valor predeterminado que se devuelve cuando una operación falla.

- Cuando se debe lanzar una excepción, se devuelve **nil**.
- Cuando se debe devolver un valor no válido, se devuelve **nil**.
- Cuando se accede a una clave inexistente en un hash, se devuelve **nil**.
- Cuando un método no devuelve explícitamente ningún valor, implícitamente devuelve **nil**.

Ejercicio 8

Determine la semántica de `null` y `undefined` para valores en javascript. ¿Qué diferencia hay entre ellos?

null: es un valor primitivo que representa la ausencia de cualquier valor o referencia a un objeto. Es considerado un objeto (valor de tipo `Object`).

- Para indicar explícitamente que una variable no tiene ningún valor o que está vacía, se le puede asignar **null**.
- Para indicar la ausencia de un objeto en contextos donde se espera un objeto, se puede utilizar **null**.

undefined: es un valor primitivo que se asigna automáticamente a una variable cuando no se le ha asignado ningún valor.

- Cuando se busca un elemento fuera del rango en un array, se devuelve **undefined**.
- Cuando se accede a una propiedad de un objeto que no existe, se devuelve **undefined**.
- Cuando un valor no ha sido definido o no tiene valor asignado, al utilizar el operador `typeof` se devuelve **undefined**.

La principal diferencia es que **null** se utiliza para representar explícitamente la ausencia de un valor asignado, mientras que **undefined** se utiliza para indicar que una variable no tiene ningún valor asignado o que una propiedad no existe.

Ejercicio 9

Determine la semántica de la sentencia `break` en C, PHP, javascript y Ruby. Cite las características más importantes de esta sentencia para cada lenguaje

C

- Se utiliza **break** dentro de bucles `'for'`, `'while'` o `'do-while'` para salir del bucle antes de que se cumpla la condición de salida normal.
- Cuando se ejecuta **break**, el control del programa sale inmediatamente del bucle más interno en el que se encuentra.
- No se puede usar **break** fuera de un bucle.

```
1  for (int i = 0; i < 10; i++) {
2      if (i == 5) {
3          break;
4      }
5      printf("%d ", i);
6  }
7  // Salida: 0 1 2 3 4
```

PHP

- Se utiliza **break** dentro de bucles 'for', 'while', 'do-while' o 'switch' para salir del bucle antes de que se cumpla la condición de salida normal.
- Cuando se ejecuta **break**, el control del programa sale inmediatamente del bucle más interno en el que se encuentra.
- Se puede especificar un número después de break para salir de varios niveles de bucles anidados.

```

1  for ($i = 0; $i < 5; $i++) {
2      if ($i === 3) {
3          break;           //Salir del bucle externo
4      }
5      for ($j = 0; $j < 3; $j++) {
6          if ($j === 1 && $i === 2) {
7              break 2;     // Salir de ambos bucles
8          }
9      }
10 }

```

JavaScript

- Se utiliza **break** dentro de bucles 'for', 'while', 'do-while' o 'switch' para salir del bucle antes de que se cumpla la condición de salida normal.
- Cuando se ejecuta **break**, el control del programa sale inmediatamente del bucle más interno en el que se encuentra.

```

1  for (let i = 0; i < 10; i++) {
2      if (i === 5) {
3          break;
4      }
5      console.log(i);
6  }
7  // Salida: 0 1 2 3 4

```

Ruby

- Se utiliza **break** dentro de bucles 'while', 'until', 'for' o 'loop' para salir del bucle antes de que se cumpla la condición de salida normal.
- Cuando se ejecuta **break**, el control del programa sale inmediatamente del bucle más interno en el que se encuentra.
- También se puede especificar un valor opcional después de **break** para devolver ese valor como resultado de la expresión que contiene el bucle.

```

1  i = 0
2  while i < 10 do
3      break if i == 5
4      puts i
5      i += 1
6  end
7  # Salida: 0 1 2 3 4

```

Ejercicio 10

Defina el concepto de ligadura y su importancia respecto de la semántica de un programa. ¿Qué diferencias hay entre ligadura estática y dinámica? Cite ejemplos (proponer casos sencillos)

Es el proceso en el que a un atributo de una entidad se le asocia un valor.

Ligadura estática:

- La asociación entre el nombre y el valor se realiza en tiempo de compilación.
- En cuanto a la estabilidad, este tipo de ligadura no se puede modificar.

```

1  // C
2  int main() {
3      static int x = 5;
4      printf("El valor de x es %d", x);
5      return 0;
6  }
7  // Salida: 5

```

Ligadura dinámica:

- La asociación entre el nombre y el valor se realiza en tiempo de ejecución.
- En cuanto a la estabilidad, este tipo de ligadura se puede modificar.
 - Existe una excepción a esta regla, las constantes.

```

1  def example():
2      x = 5
3      print("El valor de x dentro de la función es:", x)
4
5  x = 10
6  example() # Salida: El valor de x dentro de la función es: 5
7  print("El valor de x fuera de la función es:", x) # Salida: El valor de x fuera de la función es: 10

```

La variable x está ligada dinámicamente a diferentes valores en diferentes contextos. Dentro de la función example(), x está ligada al valor 5, mientras que fuera de la función example(), x está ligada al valor 10.

Práctica 4 - Variables

Objetivo: Conocer el manejo de identificadores en memoria y como lo definen e implementan los diferentes lenguajes.

Ejercicio 1:

a) Tome una de las variables de la línea 3 del siguiente código e indique y defina cuales son sus atributos:

```
1  Procedure Practica4();
2  var
3      a,i:integer
4      p:puntero
5  Begin
6      a:=0;
7      new(p);
8      p:= ^i
9      for i:=1 to 9 do
10         a:=a+i;
11     end;
12     ...
13     p:= ^a;
14     ...
15     dispose(p);
16 end;
```

b) Compare los atributos de la variable del punto a) con los atributos de la variable de la línea 4. ¿Qué dato contiene esta variable?

a) a

Nombre = a

Alcance = líneas 3 - 16, local

Tipo = predefinido integer, momento de ligadura estático, explícita

l-valor = tiempo de vida 1 - 16, alocaión dinámica automática

r-valor = dinámico, línea 3 = basura

b) p

Nombre = p

Alcance = líneas 4 - 16, local

Tipo = definido por el usuario puntero, momento de ligadura estático, explícita

l-valor = tiempo de vida 1 - 16, alocaión dinámica explícita

r-valor = dinámico, línea 8 = l-valor de i, línea 13 = l-valor de a

Ejercicio 2

- a. Indique cuales son las diferentes formas de inicializar una variable en el momento de la declaración de la misma.
- b. Analice en los lenguajes: Java, C, Python y Ruby las diferentes formas de inicialización de variables que poseen. Realice un cuadro comparativo de esta característica.

a) Estrategias de inicialización:

- Inicialización por defecto:
 - Enteros se inicializan en 0.
 - Caracteres en blanco.
 - Funciones en VOID.
- Inicialización en la declaración:
 - `int x = 5;`
- Ignorar
 - Toma como valor lo que hay en la celda de memoria.

b)

Lenguaje	Declaración e inicialización en misma línea.	Declaración e inicialización en distinta línea.	Inicialización por defecto
Java	Sí	Sí	Sí
C	Sí	Sí	Sí para globales y estáticas No para locales
Python	Sí	Sí <code>x = None</code> (ausencia de valor)	No
Ruby	Sí	Sí <code>x = nil</code> (ausencia de valor)	No

Ejercicio 3

Explique los siguientes conceptos asociados al atributo l-valor de una:

- a. Variable estática.
- b. Variable automática o semiestática.
- c. Variable dinámica.
- d. Variable semidinámica.

De al menos un ejemplo de cada uno.

Investigue sobre qué tipos de variables respecto de su l-valor hay en los lenguajes C y Ada.

a) Variable estática

La alocaión se hace en compilación (previo a ejecución) cuando se carga el programa en memoria en zona de datos y perdura hasta el fin de la ejecución. El único lenguaje con esta característica dado en la cátedra es C.

```
1 // C
2 void method() {
3     static int = 0;
4     printf("%d ", num);
5     num++;
6 }
7
8 int main() {
9     for (int i = 0; i < 5; i++) {
10         method();
11     }
12     return 0;
13 }
14 // Salida: 0 1 2 3 4
```

b) Variable automática o semiestática.

La alocaión se hace en tiempo de ejecución, cuando aparece una declaración. También conocida como variable local, su ciclo de vida está limitado al ámbito en el que se declara. Se asigna en memoria cuando entra al ámbito y se libera cuando sale de él.

```
1 # Python
2 def method():
3     num = 0
4     print(num, " ")
5     num += 1
6
7 for i in range(5):
8     method()
9
10 # Salida: 0 0 0 0 0
```

c) Variable dinámica

La asignación se hace en tiempo de ejecución, se crea y se destruye. Se asigna y libera memoria dinámicamente según sea necesario. Estas variables se crean utilizando funciones específicas como new() o input().

```
1  // Pascal
2  procedure method();
3  var
4      num:integer;
5  begin
6      num := read();
7      write(num, ' ');
8  end;
9  var
10     i : integer;
11  Begin
12     for i:=1 to 5 do
13         method();
14  End.
15  // Salida: depende del usuario
```

d) Variable semidinamica

La asignación se hace en tiempo de ejecución. Es aquella cuyo tamaño puede cambiar durante la ejecución del programa.

Ada:

- **Variable estática:** se definen usando la palabra clave “constant” o “static” y se mantiene en memoria durante toda la ejecución del programa.
- **Variable automática o semiestática:** se definen dentro de un bloque y se eliminan de la memoria cuando el bloque termina.
- **Variable dinámica:** se asignan en tiempo de ejecución usando la palabra clave “new”. Estas variables se mantienen en la memoria hasta que se libera explícitamente usando la palabra clave “delete”.
- **Variable semidinámica:** una matriz cuyo tamaño se puede ajustar en tiempo de ejecución, pero sólo en un punto de control específico del programa.

C

- **Variable estática:** son variables que se almacenan en la memoria estática y mantienen su valor entre llamadas a funciones. Estas variables se declaran con la palabra clave “static” y se inicializan automáticamente en cero.
- **Variable automática:** son variables que se almacenan en la memoria de la pila y se eliminan automáticamente al salir de su ámbito de definición. Estas variables se declaran sin la palabra clave “static” y su vida útil está limitada a su función o bloque de código.
- **Variable dinámica:** son variables que se asignan en tiempo de ejecución y se almacenan en la memoria dinámica. Estas variables se crean utilizando las funciones “malloc” o “calloc” y se liberan utilizando la función “free”. Estas variables

tienen una vida útil que se extiende más allá de la función o bloque de código en el que se crean.

Ejercicio 4

- a. ¿A qué se denomina variable local y a qué se denomina variable global?
- b. ¿Una variable local puede ser estática respecto de su l-valor? En caso afirmativo dé un ejemplo
- c. Una variable global ¿siempre es estática? Justifique la respuesta.
- d. Indique qué diferencia hay entre una variable estática respecto de su l-valor y una constante

a)

- **Variable local:** variable que se ha creado dentro de una unidad (programa o subprograma).
- **Variable global:** variable que se ha creado en el programa principal.

b) Sí. En C se puede definir como **static** una variable local para mantener su valor entre llamadas sucesivas a una función.

```
1  // C
2  void method() {
3      static int numLocal = 1;
4      printf("%d ", numLocal);
5      numLocal++;
6  }
7
8  int main() {
9      for (int i=0; i<6; i++) {
10         method();
11     }
12 }
13 // Salida: 1 2 3 4 5
```

c) **Una variable global ¿siempre es estática?**

No necesariamente. La estaticidad es una propiedad que puede tener una variable independientemente de su alcance. En algunos lenguajes de programación, como C, las variables globales son, por defecto, estáticas en términos de su l-valor, lo que significa que su valor se mantiene durante toda la vida útil del programa y no puede ser modificado por otras funciones o bloques de código. Sin embargo, en otros lenguajes de programación como Java y Python, las variables globales no son estáticas en términos de su l-valor, ya que pueden ser modificadas en tiempo de ejecución y su valor puede ser accedido y modificado por cualquier función o bloque de código en el programa.

d) **Diferencia entre una variable estática respecto de su l-valor y una constante**

La diferencia principal entre una variable estática respecto a su l-valor y una constante es que la variable estática es alocada en tiempo de compilación mientras que la constante en tiempo de ejecución, cuando su unidad es alocada. Existe una excepción para esta regla, las constantes numéricas de Ada, que también son alocadas en compilación

Además, el tiempo de vida de las variables estáticas sobrepasan el de las constantes, porque las variables estáticas sobrepasan el tiempo de vida del programa.

Ejercicio 5

- a. En Ada hay dos tipos de constantes, las numéricas y las comunes. Indique a qué se debe dicha clasificación.
- b. En base a lo respondido en el punto a), determine el momento de ligadura de las constantes del siguiente código:

```
H: constant Float:= 3,5;  
I: constant:= 2;  
K: constant float:= H*I;
```

- a) Las constantes comunes son las constantes a las que se les define el tipo (H y K), y las numéricas son las que no tienen el tipo definido (I). Estas se diferencian porque a las numéricas se les asigna el r-valor en tiempo de compilación, y a las comunes en ejecución.
- b) La ligadura de las variables **H** y **K** se realiza en tiempo de ejecución, ya que estas son constantes numéricas. Por otro lado, la ligadura de la variable **I** se realiza en tiempo de compilación.

Ejercicio 6

Sea el siguiente archivo con funciones de C:

```
1  {  
2      int x=1; (1)  
3      int func1();{  
4          int i;  
5          for (i:=0; i < 4; i++) x=x+1;  
6      }  
7      int func2();{  
8          int i, j;  
9          /*sentencias que contienen declaraciones y  
10         sentencias que no contienen declaraciones*/  
11         .....  
12         for (i:=0; i < 3; i++) j=func1 + 1;  
13     }  
14 }
```

Analice si llegaría a tener el mismo comportamiento en cuanto a alocaión de memoria, sacar la declaración (1) y colocar dentro de **func1()** la declaración **static int x =1;**.

Sí, tendría el mismo comportamiento, ya que si se tiene **static int x=1** en la **func1()**, la memoria se asignaría una sola vez cuando se llama por primera vez a la función y esta se mantendría asignada durante todo el programa, así como pasa con la variable global **x** en este caso.

Ejercicio 7

Sea el siguiente segmento de código escrito en Java, indique para los identificadores si son globales o locales.

```
Clase Persona {
    public long id;                // "Global"
    public string nombreApellido;  // "Global"
    public Domicilio domicilio;    // "Global"
    private string dni;            // Local
    public string fechaNac;        // "Global"
    public static int cantTotalPersonas; // "Global"

    //Se tienen los getter y setter de cada una de las variables

    // Este método calcula la edad de la persona a partir de la fecha de nacimiento
    public int getEdad(){
        public int edad=0;        // Local
        public string fN =        // Local
            this.getFechaNac();
        ...
        ...
        return edad;
    }
}

Clase Domicilio {
    public long id;                // "Global"
    public static int nro;          // "Global"
    public string calle;            // "Global"
    public Localidad loc;          // "Global"

    //Se tienen los getter y setter de cada una de las variables
}
```

En Java no existe el concepto de variable global, en su lugar las variables en Java se definen dentro de clases y pueden tener diferentes niveles de alcance. Se puede simular el comportamiento de variable global en Java utilizando variables públicas y públicas estáticas.

Ejercicio 8

Sea el siguiente ejercicio escrito en Pascal

```
1  Program Uno;
2  type tpuntero= ^integer;
3  var mipuntero: tpuntero;
4  var i:integer;
5  var h:integer;
6  Begin
7      i:=3;
8      mipuntero:=nil;
9      new(mipuntero);
10     mipuntero^:=i;
11     h:= mipuntero^+i;
12     dispose(mipuntero);
13     write(h);
14     i:= h- mipuntero;
15 End.
```

- a) Indique el rango de instrucciones que representa el tiempo de vida de las variables i, h y mipuntero.
- b) Indique el rango de instrucciones que representa el alcance de las variables i, h y mipuntero.
- c) Indique si el programa anterior presenta un error al intentar escribir el valor de h. Justifique
- d) Indique si el programa anterior presenta un error al intentar asignar a i la resta de h con mipuntero. Justifique
- e) Determine si existe otra entidad que necesite ligar los atributos de alcance y tiempo de vida para justificar las respuestas anteriores. En ese caso indique cuál es la entidad y especifique su tiempo de vida y alcance.
- f) Especifique el tipo de variable de acuerdo a la ligadura con el l-valor de las variables que encontró en el ejercicio.

a) i: 1 - 15
h: 1 - 15
mipuntero: 1 - 15
mipuntero^: 9 - 12

b) i: 5- 15
h: 6 - 15
mipuntero: 4- 15
mipuntero^: 4- 15

- c) No, porque el valor de **h** es asignado previo al **dispose(mipuntero)**.
- d) Sí, esto presenta un error porque se intenta realizar una operación de resta entre un **integer(h)** y un tipo de dato definido por el usuario(**mipuntero**).
- e) Sí, la otra entidad es el programa principal. En Pascal, tiene alcance global y su tiempo de vida es desde su declaración hasta el final del programa (6 - 15).
- f) i: entero
h: entero
mipuntero: puntero a entero (definido por usuario)

Ejercicio 9

Elija un lenguaje y escriba un ejemplo:

- a. En el cual el tiempo de vida de un identificador sea mayor que su alcance

```

1 // C
2 void method() {
3     static int num = 1;
4     printf("%d ", num);
5     num++;
6 }
7
8 int main() {
9     for (int i=0; i<6; i++) {
10         method();
11     }
12 }
13 // Salida: 1 2 3 4 5

```

El alcance de la variable **num** es solo dentro de la función **method()**, pero su tiempo de vida es desde que se compila el programa hasta que este finaliza, ya que se trata de una variable estática.

- b. En el cual el tiempo de vida de un identificador sea menor que su alcance

```

1 // Pascal
2 program ejemploB;
3 type
4     tipo_puntero = ^integer;
5 var
6     puntero: tipo_puntero;
7 Begin
8     new(puntero);
9     puntero^ := 10;
10    dispose(puntero);
11    write(puntero^);
12 End.

```

El alcance de la variable **puntero^** es desde la línea 7 (declaración) hasta la 12 (fin del bloque), pero su tiempo de vida es desde la línea 8 (**new()**) hasta la 10 (**dispose()**).

- c. En el cual el tiempo de vida de un identificador sea igual que su alcance
Nunca va a ocurrir esto, ya que el alcance de una variable empieza a contar una línea después de su declaración. Por lo tanto, aunque declaremos una variable en la primera línea de un programa de 15 líneas, su alcance será desde la línea 2 hasta la 15, y su tiempo de vida desde la 1 hasta la 15.

Ejercicio 10

Si tengo la siguiente declaración al comienzo de un procedimiento:

int c; **en C**

var c:integer; **en Pascal**

c: integer; **en ADA**

Y ese procedimiento NO contiene definiciones de procedimientos internos. ¿Puedo asegurar que el alcance y el tiempo de vida de la variable “c” es siempre todo el procedimiento en donde se encuentra definida?. Analícelo y justifique la respuesta, para todos los casos.

Sí, en los tres lenguajes se puede asegurar que la variable **c** tiene alcance y tiempo de vida limitados al procedimiento en el que se encuentra definida, ya que no hay definiciones de procedimientos internos que puedan afectar a su alcance o tiempo de vida

Ejercicio 11

- a) Responda Verdadero o Falso para cada opción.

¿El tipo de dato de una variable es?

- i) Un string de caracteres que se usa para referenciar a la variable y operaciones que se pueden realizar sobre ella. **F**
- ii) Conjunto de valores que puede tomar y un rango de instrucciones en el que se conoce el nombre. **F**
- iii) Conjunto de valores que puede tomar y lugar de memoria asociado con la variable. **F**
- iv) Conjunto de valores que puede tomar y conjunto de operaciones que se pueden realizar sobre esos valores. **V**

- b) Escriba la definición correcta de tipo de dato de una variable.

Ejercicio 12

Sea el siguiente programa en ADA, completar el cuadro siguiente indicando para cada variable de que tipo es en cuanto al momento de ligadura de su l-valor, su r-valor al momento de aloación en memoria y para todos los identificadores cuál es su alcance y cual es su el tiempo de vida.

Indicar para cada variable su r-valor al momento de aloación en memoria

```

1. with text_io; use text_io;
2. Procedure Main is;
3.   type vector is array(integer range <>);
4.   a, n, p:integer;
5.   v1:vector(1..100);
6.   c1: constant integer:=10;
7.   Procedure Uno is;
8.     1. type puntero is access integer;
9.     2. v2:vector(0..n);
10.    3. c1, c2: character;
11.    4. p,q: puntero;
12.    5. begin
13.      7.5.1. n:=4;
14.      7.5.2. v2(n):= v2(1) + v1(5);
15.      7.5.3. p:= new puntero;
16.      7.5.4. q:= p;
17.      7.5.5. ....
18.      7.5.6. free p;
19.      7.5.7. ....
20.      7.5.8. free q;
21.      7.5.9. ....
22.    7.6. end;
23. begin
24. 9. n:=5;
25. 10. ....
26. 11. U7no;
27. 12. a:= n + 2;
28. 13. ....
29. 14. end

```

Ident.	Tipo (l-valor)	r-valor	Alcance	T.V
Main ()			3 - 14	1 - 14
a (4)	automática	basura	5 - 14	1 - 14
n (4)	automática	basura	5 - 14	1 - 14
p (4)	automática	basura	5 - 7.4 7.6 - 14	1 - 14
v1 (5)	automática	basura	6 - 14	1 - 14
c1 (6)	automática	10	7 - 7.3 7.6 - 14	1 - 14
Uno (7)			7.1 - 14	7 - 7.6
v2 (7.2)	semidinámica	basura	7.3 - 7.6	7 - 7.6
c1 (7.3)	automática	basura	7.4 - 7.6	7 - 7.6
c2 (7.3)	automática	basura	7.4 - 7.6	7 - 7.6
p (7.4)	automática	nil	7.5 - 7.6	7 - 7.6
q (7.4)	automática	nil	7.5 - 7.6	7 - 7.6
p^ (7.5.3)	dinámica	basura	7.5 - 7.6	7.5.4-7.5.6
q^ (7.5.4)	dinámica	basura	7.5-7.6	7.5.5-7.5.8

Aclaración:

Ident.= Identificador

Tipo = tipo de la variable respecto del l-value

r-valor = debe ser tomado al momento de la aloación en memoria.

T.V. = Tiempo de Vida

El alcance de los identificadores debe indicarse desde la línea siguiente a su declaración.

Ejercicio 13

El nombre de una variable puede condicionar:

- a) **Su tiempo de vida:** No, el tiempo de vida de una variable no depende de su nombre, depende de la unidad en la que esté declarada, o en el caso de las dinámicas, de los new() y dispose().
- b) **Su alcance:** Si, el nombre puede llegar a condicionar el alcance de una variable, ya que si dos variables se llaman igual, la que haya sido declarada primera va a ser enmascarada por la segunda, haciendo que su alcance disminuya.
- c) **Su r-valor:** No, el r-valor depende de la declaración de la variable, al menos en tiempo de compilación, en la mayoría de lenguajes este comienza siendo basura, pero en el caso de C, las variables estáticas y las globales son inicializadas.
- d) **Su tipo:** No, el tipo solo depende de la declaración, ya sea explícita o implícita. Esto quiere decir que solo va a depender del tipo o del valor que se le asigne.

Justifique la respuesta

Ejercicio 14

Sean los siguientes archivos en C, los cuales se compilan juntos

Indicar para cada variable de qué tipo es en cuanto al momento de ligadura de su l-valor.

Indicar para cada identificador cuál es su alcance y cual es su el tiempo de vida.

Indicar para cada variable su r-valor al momento de alocaación en memoria

ARCHIVO1.C 1. int v1; 2. int *a; 3. Int fun2 () 4. { int v1, y; 5. for(y=0; y<8; y++) 6. { extern int v2; 7. ...} 8. } 9. main() 10. { static int var3; 11. extern int v2; 12. int v1, y; 13. for(y=0; y<10; y++) 14. { char var1='C'; 15. a=&v1;} 16. } ARCHIVO2.C 17. static int aux; 18. int v2; 19. static int fun2() 20. { extern int v1;					
	Ident.	Tipo	r-valor	Alcance	T.V
	v1 (1)	automática	0	2 - 4 9 - 12 21 - 23	1 - 28
	a (2)	automática	null	3 - 16	<1 - 28>
	*a (2)	dinámica	basura	3 - 16	15 - 15
	fun2 (3)			4 - 16	3 - 8
	v1 (4)	automática	basura	5 - 8	3 - 8
	y (4)	automática	basura	5 - 8	3 - 8
	main ()			10 - 16	9 - 16
	var3 (10)	estática	0	11 - 16	<1 - 28>
	v1 (12)	automática	basura	13 - 16	9 - 16
	y (12)	automática	basura	13 - 16	9 - 16

21. aux=aux+1;	var1 (14)	automática	basura	15 - 15	13 - 15
22. ...	aux (17)	estática	0	18 - 25	<1 - 28>
23. }	v2 (18)	automática	0	7 - 7 12 - 16 19 - 28	1 - 28
24. int fun3()	fun2 (19)			20 - 28	19 - 23
25. { int aux;	fun3 (24)			25 - 28	24 - 28
26. aux=aux+1;	aux (25)	automática	basura	26 - 28	24 - 28
27. ...					
28. }					

Aclaración:

Ident. = Identificador

Tipo = tipo de la variable respecto del l-value

r-valor = debe ser tomado al momento de la alocaión en memoria.

T.V. = Tiempo de Vida

El alcance de los identificadores debe indicarse desde la línea siguiente a su declaración.

Ejercicio 15

Para javascript investigue la diferencia semántica para declarar una variable utilizando los modificadores const, var, let y la ausencia de cualquiera de estos. Compárelo con un lenguaje de su preferencia.

Hoisting: es un mecanismo de JavaScript donde las variables y las declaraciones (solo declaraciones, no inicialización) de funciones se mueven a la parte superior de su alcance antes de la ejecución del código

- **var**

- El alcance es global cuando una variable var se declara fuera de una función. Esto significa que cualquier variable que se declare con var fuera de un bloque de funciones está disponible para su uso en todo el programa.
- var tiene un alcance de función cuando se declara dentro de una función. Esto significa que está disponible y se puede acceder solo dentro de esa función.
- Las variables declaradas con var se pueden volver a declarar y actualizar
- Hoisting de var:
 - Las variables var se elevan a la parte superior de su alcance y se inicializan con un valor de indefinido.

- **let**

- Una variable declarada en un bloque con let solo está disponible para su uso dentro de ese bloque (Un bloque es un fragmento de código delimitado por {}). Un bloque vive entre llaves. Cualquier cosa entre llaves es un bloque)

- let se puede actualizar, pero no volver a declarar (si la misma variable se define en diferentes ámbitos, no habrá error).
- Hoisting de let:
 - Las variables let se elevan a la parte superior de su alcance pero no se inicializan.
- **const**
 - Las variables declaradas con **const** mantienen valores constantes.
 - Solo se puede acceder a las declaraciones **const** dentro del bloque en el que se declararon.
 - **const** no se puede actualizar o volver a declarar.
 - Cada declaración con **const** debe inicializarse en el momento de la declaración.
 - Hoisting de const:
 - Las variables **const** se elevan a la parte superior de su alcance pero no se inicializan.

Si se declara una variable sin utilizar const, let o var, el motor de JavaScript va a intentar buscar la variable en un contexto superior de forma recursiva. Para ello, primero busca en el ámbito local, si no la encuentra, se intenta en su contexto inmediatamente superior, y así hasta que se llegue al contexto global. Si no se ha encontrado en ningún contexto, esa variable se crea automáticamente y su comportamiento es similar al de var en cuanto a su alcance y mutabilidad. El problema es si esa variable se encuentra en uno de esos contextos superiores que se han ido comprobando, ya que tomará su valor.

En comparación, en Python, no se utilizan modificadores de este tipo para declarar variables. En Python, las variables se declaran simplemente asignándoles un valor. Además, el alcance de las variables se define por la indentación, es decir, por la estructura del código. Por ejemplo, una variable declarada dentro de una función sólo es accesible dentro de esa función. Si se intenta acceder a ella fuera de la función, se producirá un error.

Práctica 5 - Pilas de ejecución

Objetivo: Interpretar cómo se organiza la memoria de datos durante la ejecución de un programa con llamados a subrutinas.

Ejercicio 1

Explique claramente cual es la utilidad del registro de activación y que representan cada una de sus partes. Basado en el modelo debajo detallado.

Head (programa principal)
Punto de retorno (PR)
Enlace estático (EE)
Enlace dinámico (ED)
Variables
...
Parámetros
...
Procedimientos
...
Funciones
...
Valor de retorno

- Head
 - Current: dirección base del registro de activación de la unidad que se esté ejecutando actualmente.
 - Free: próxima dirección libre en la pila.
- Punto de retorno: la siguiente dirección a ejecutar después de que termine la subrutina.
- Enlace estático: apunta al registro de activación de la unidad que estáticamente la contiene.
- Enlace dinámico: contiene un puntero a la dirección base del registro de activación de la rutina llamadora.
- Variables: las variables definidas dentro de la unidad.
- Procedimientos: los procedimientos definidos dentro de la unidad.
- Funciones: las funciones definidas dentro de la unidad.
- Valor de retorno: al terminar una rutina se desaloca su registro de activación, por lo tanto la rutina llamante debe guardar en su RA el valor de retorno de la rutina llamada.

Ejercicio 2

Dado el siguiente programa escrito en Pascal-like, continuar la realización de las pilas de ejecución hasta finalizar las mismas.

- a) Siguiendo la cadena estática
- b) Siguiendo la cadena dinámica

<pre>Program Main Var a: array[1..10] of integer; x,y,z:integer Procedure A () var y,t: integer; begin a(1):= a(1)+1;z:=z+1; t:=1; y:=2; B(); a(y):=a(y)+3; y:=y+1; If z=11 then begin a(z-1):=a(z-2) + 3; z:=z-4; a(z-y):=a(z) - a(y) + 5; end; end; Function t():integer begin y:=y+1; z:=z-6; return(y+x); end;</pre>	<pre>Procedure B() var d:integer; Procedure I () begin x:=0; x:=x+6; end; begin x:=x+t; d:=0; while x>d do begin I(); x:=x-1; d:=d + 2; end; end; Begin For x:=1 To 10 do a(x):=x; x:=5; y:=1; z:=10; A(); For x:=1 To 10 do write(a(x),x); end.</pre>
--	---

Nota: La forma de evaluación de este lenguaje es de izquierda a derecha

Siguiendo la cadena estática

	*** Reg Activ Main
*1	Pto retorno
	A(1) = 4
	A(2) = 2
	A(3) = 3
	A(4) = 4
	A(5) = 5
	A(6) = 6
	A(7) = 7
	A(8) = 8
	A(9) = 9
	A(10) = 10
	X = 4 - 10 - 5
	Y = 4 - 2
	Z = 40 - 44 - 5
	Procedure A
	Function T
	Procedure B
	VR
*2	***Reg Activ A
	Pto retorno
	EE (*1)
	EE (*1)
	Y = 2
	T = 1
	VR
	***Reg Activ B
	Pto retorno
	EE
	ED
	D =
	Procedure I
	VR
	***Reg Activ ... (a partir de acá lo debe continuar...)
	

Siguiendo la cadena dinámica

	*** Reg Activ Main
*1	Pto retorno
	A(1) = 4, 2, 5
	A(2) = 2
	A(3) = 3
	A(4) = 4
	A(5) = 5
	A(6) = 6
	A(7) = 7
	A(8) = 8
	A(9) = 9
	A(10) = 10
	X = 4 - 10 5
	Y = 4 - 2
	Z = 40 - 11
	Procedure A
	Function T
	Procedure B
	VR
*2	***Reg Activ A
	Pto retorno
	EE (*1)
	EE (*1)
	Y = 2
	T = 1
	VR
*3	***Reg Activ B
	Pto retorno
	EE (*1)
	ED (*2)
	D =
	Procedure I
	VR
*4	***Reg Activ ... (a partir de acá lo debe continuar...)
	

a)

Current		
2)	Free	VR
	*** Reg Activ Main	*** Reg Activ B
*1	PR	*3 PR
	EE	EE (*1)
	ED	ED (*2)
	a(1) = 4.2	d = 0246
	a(2) = 2.5	Procedure I
	a(3) = 3	VR 7
	a(4) = 4	*** Reg Activ E
	a(5) = 5	*4 PR
	a(6) = 6	EE (*1)
	a(7) = 7	ED (*3)
	a(8) = 8	VR
	a(9) = 9	*5 *** Reg Activ I
	a(10) = 10	PR
	x = 1...10 5 12 16 50 650	EE (*3)
	y = 7 2	ED (*3)
	z = 10 11 5	VR
	Procedure A	*6 RA I
	Function t	PR
	Procedure B	EE (*3)
	VR	ED (*3)
	*** Reg Activ A	VR
*2	PR	*7 RA I
	EE (*1)	PR
	ED (*1)	EE (*3)
	y = 2.3	ED (*3)
	t = 1	VR

b)

Current	*3 RA B
Free	PR
RA Main	EE(*1)
*1 PR	ED(*2)
$a(1) = 12$	$d = 0246$
$a(2) = 85$	Procedure I
$a(3) = 3$	VR
$a(4) = 49$	RA I
$a(5) = 5$	*4 PR
$a(6) = 6$	EE(*3)
$a(7) = 7$	ED(*3)
$a(8) = 8$	VR
$a(9) = 9$	RA I
$a(10) = 1012$	*5 PR
$x = 1 \dots 10560650650657 \dots 10$	EE(*3)
$y = 1$	ED(*3)
$z = 10117$	VR
Procedure A	RA I
Function t	*6 PR
Procedure B	EE(*3)
VR	ED(*3)
RA A	VR
*2 PR	
EE(*1)	
ED(*1)	
$y = 23$	
$t = 1$	
VR	

Ejercicio 3

Sea el siguiente programa escrito en Pascal-like. Realice la pila de ejecución

- a) Siguiendo la cadena estática
- b) Siguiendo la cadena dinámica

<pre>PROGRAM P1; var a:integer; b:char; c: array[1..10] of integer Procedure PP1; var a:char; p:integer; Function x: integer; var z:integer; begin a:="j"; z=-1; return z; end; Begin p:=x; write(a); p:=x+3; c[p]=8; p:=x+2; c[p]=x; end;</pre>	<pre>Procedure x; var b:char; Procedure PP2; Begin write("para qué estoy aquí?"); end; Begin a:=1; c[a]:=4; b:="a"; write(concat(c[1],b)); /*concat convierte a string los parámetros, concatena y retorna un string;*/ PP1(); b:="b"; write(concat(c[5],b)); /*concat convierte a string los parámetros, concatena y retorna un string;*/ End; BEGIN a:=3; b:="c"; for a:=3 to 10 do begin c[a]:=2*a; end; x; write(b); write(a); for a:=1 to 10 do write(c[a]-3); END.</pre>
--	--

Nota: La forma de evaluación de este lenguaje es de izquierda a derecha

a) y b) quedan iguales.

3) Current	$p = -7 \neq 1$
File	Function x
Reg Activ P1	VR $\cancel{1} \cancel{2} \cancel{3} \cancel{4}$
*1 PR	RA x
$a = 3 \cancel{3} \dots \cancel{10} \cancel{4} \cancel{7} \dots \cancel{10}$	*4 PR
$b = "c"$	EE(*3)
$c(1) = 4 - 1$	ED(*3)
$c(2) = 8$	$z = -1$
$c(3) = 6$	VR
$c(4) = 8$	RA x
$c(5) = 10$	*5 PR
$c(6) = 12$	EE(*3)
$c(7) = 14$	ED(*3)
$c(8) = 16$	$z = -1$
$c(9) = 18$	VR
$c(10) = 20$	RA x
Procedure PP1	*6 PR
Procedure x	EE(*3)
VR	ED(*3)
RA x	$z = -1$
*2 PR	VR
EE(*1)	RA x
ED(*1)	*7 PR
$b = "a" "b"$	EE(*3)
Procedure PP2	ED(*3)
VR	$z = 1$
RA PP1	VR
*3 PR	
EE(*1)	
ED(*2)	
$a = "j" "i" "j" "i"$	

Ejercicio 4

Sea el siguiente programa escrito en Pascal-like. Realice la pila de ejecución

- a) Siguiendo la cadena estática
- b) Siguiendo la cadena dinámica

```
Procedure Main;
  var x, y: integer;
  vec: array[1..7] of integer;
  Function B:integer;
    var y:integer;
    begin
      y:=4; x:= y - 2;
      return (x);
    end;
  Procedure D;
    var i, x: integer;
    vec: array[1..7] of integer;
    Procedure A;
      var y:integer;
      begin
        y:=x + 5; vec(i+2):=vec(i + 2) + y;
        x:= x +B; C;
      end;
    Function B:integer;
      begin
        vec(i):= y + 2; i:=i+2;
        vec(i):= vec(1) * i;
        return ( vec(i)-vec(1) );
      end;
    begin
      for x:= 1 to 7 do vec(x):= 1;
      x:=1; i:= 2;
      if y = 7 then A; else C;
      for x:= 1 to 7 do write(vec(x));
    end;
```

```
Procedure C;
  var i, y: integer;
  begin
    i:= 1; y:= 6; x:= x + B;
    vec(2):= vec(2) * x;
    while (i < y) do begin
      vec(i):= vec(i) + B - 1;
      i:= i + 3;
    end;
    y:= y - 4;
  end;
begin
  for x:= 1 to 7 do vec(x):= x;
  x:= 3; y:= B+5; D;
  if (x = 2) then begin
    vec(x):= vec(x) + 2;
    vec(x + 3):= vec(x) * 3;
  end;
  for x:= 1 to 7 do write(vec(x));
end.
```

Nota: La forma de evaluación de este lenguaje es de izquierda a derecha

a)

4) Current
Free
RA Main
*1 PR
x = 1...7 3 2 2 4 2 2 1...7
y = 7
vec(1) = 12
vec(2) = 2 2 70
vec(3) = 3
vec(4) = 45
vec(5) = 5 30
vec(6) = 6
vec(7) = 7
Function B
Procedure D
Procedure C
VR 2
RA B
2 PR
EE(*1)
ED(*1)
y = 4
VR
RA D
3 PR
EE(*1)
ED(*1)
i = 24
x = 1...7 7 4 7...7
vec(1) = 1
vec(2) = 19
vec(3) = 1
vec(4) = 7 7 4

vec(5) = 1
vec(6) = 1
vec(7) = 1
Procedure A
Function B
VR
RA A
*4 PR
EE(*3)
ED(*3)
y = 6
VR 3
RA B
*5 PR
EE(*3)
ED(*4)
VR
RA C
*6 PR
EE(*1)
ED(*1)
i = 147
y = 62
VR 222
RA B
*7 PR
EE(*1)
ED(*6)
y = 4
VR
RA B
*8 PR
EE(*1)
ED(*6)
y = 4
VR
RA B
*9 PR
EE(*1)
ED(*6)
y = 4
VR
*5 PR
EE(*3)
ED(*4)
VR
RA C
*6 PR
EE(*1)
ED(*1)
i = 147
y = 62
VR 222
RA B
*7 PR
EE(*1)
ED(*6)
y = 4
VR
RA B
*8 PR
EE(*1)
ED(*6)
y = 4
VR
RA B
*9 PR
EE(*1)
ED(*6)
y = 4
VR

Papel de fibra de caña de azúcar.

Ejercicio 5

Sea el siguiente programa escrito en Pascal-like. Realice la pila de ejecución

- a) Siguiendo la cadena estática
- b) Siguiendo la cadena dinámica
- c) La sentencia $x := c + 5 + x$, podría reemplazarse por $x := x + c + 5$? Justifique la respuesta

<pre>Program Main; Var x, y, z:integer; a, b: array[1..6] of integer; Procedure B; var y,x: integer; Procedure C; var c:integer; begin y:= y + 2; c:=2; a(x):=a(x)*y; if (y >7) then b(y-6)=b(4)*2+b(y-6); D; end; begin x:=2; y:= x + 3; C; x:= x + 1; write (x,y); End; Procedure D; begin x:= c + 5 + x; y:= y + 2; end;</pre>	<pre>Function C: integer; begin b(x):= b(x) + 1; x:= x + 1; a(y):=a(y)+b(x)+3; a(x+2)=a(x) + 2; return b(x); end begin x:= 1; Y:= 2; for z:=1 to 6 do begin a(z):= z; b(z):= z + 2; end; B; for z:= to 6 do write (a(z), b(z)); end.</pre>
--	--

Nota:La forma de evaluación de este lenguaje es de izquierda a derecha

Práctica 6 - Parámetros

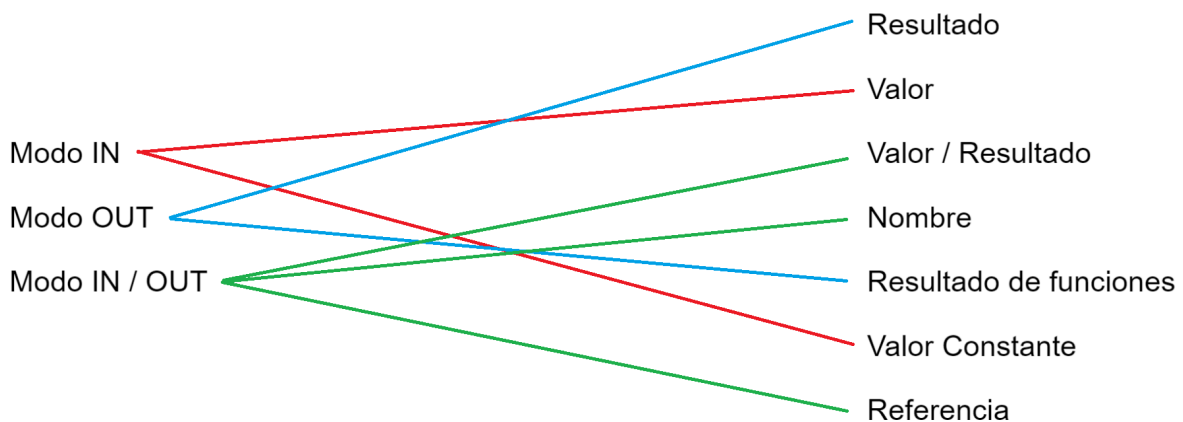
Objetivo: Descubrir las técnicas existentes para pasaje de parámetros entre unidades y sus diferencias esenciales de acuerdo al lenguaje que lo implementa

Ejercicio 1

a- Explique brevemente los siguientes conceptos

- **Parámetro:** forma de conectar y compartir datos entre unidades de programa.
- **Parámetro real:** es un valor u otra entidad que se pasa a un procedimiento o función. Son los que se colocan en la parte de la invocación de la rutina.
- **Parámetro formal:** es una variable utilizada para recibir valores de entrada en una rutina, subrutina, etc. Son los que se colocan en la parte de la declaración de la rutina.
- **Ligadura posicional:** se corresponden con la posición que ocupan en la lista de parámetros. Van en el mismo orden.
- **Ligadura por palabra clave o nombre:** se corresponden con el nombre por lo tanto pueden estar colocados en distinto orden en la lista de parámetros.

Ejercicio 2: Unir los siguientes puntos según corresponda y de una definición y un ejemplo de cada par.



❖ Modo IN

- **Valor:** el valor del parámetro real es copiado al parámetro formal al momento de llamar a la unidad. El parámetro formal actúa como una variable local de la unidad. La conexión se genera solo al inicio.
- **Valor Constante:** se envía un valor a la unidad, pero esta no puede modificarlo. Dependerá del lenguaje si se realiza una copia o no.

❖ Modo OUT

- **Resultado:** el valor del parámetro formal es copiado en el parámetro real al terminar de ejecutar la unidad. El parámetro formal es una variable local de la unidad, pero sin valor porque no recibe nada, por lo tanto debe inicializarse, si no puede traer problemas.

- **Resultado de funciones:** es el resultado que devuelven las funciones. Puede devolverse de distintas formas según el lenguaje: return, nombre de la función.

❖ **Modo IN/OUT**

- **Valor / Resultado:** el parámetro formal es una variable local a la rutina que recibe una copia de su parámetro real. Al finalizar la ejecución de la unidad el se copia el valor del parámetro formal en el parámetro real. Generando así una conexión solamente al inicio y al final.
- **Por referencia:** se asocia el l-valor del parámetro real al parámetro formal, por lo tanto, la conexión es de inicio a fin. Cada referencia al parámetro formal será a un ambiente no local, por lo que cada cambio quedará registrado en el parámetro real.
- **Nombre:** el parámetro formal es sustituido “textualmente” por una expresión del parámetro real, sumado a un puntero a la unidad del parámetro real. La primera conexión se da en la invocación de la rutina, pero cada “ligadura de valor” se da sólo cuando se utiliza el parámetro.

Ejercicio 3:

- a. Complete el siguiente cuadro según lo correspondiente a cada lenguaje:

Tipo de pasaje de parámetros	Lenguaje
IN: valor OUT: resultado IN/OUT: valor-resultado para primitivos, referencia para no primitivos.	ADA
IN: valor, valor constante	C
IN: valor Mutable: IN/OUT por referencia	Ruby
Sólo se utiliza IN por valor, pero las variables no primitivas son todas anónimas en la heap, el paso por valor se da por referencia.	Java
Inmutables: IN por valor Mutables: IN/OUT por referencia	Python

- b. Ada es más seguro que Pascal, respecto al pasaje de parámetros en las funciones. Explique por qué.

(Agus) Ada es más seguro que Pascal en cuanto al pasaje de parámetros en las funciones debido a su sistema de tipos más estricto y su sintaxis más clara y explícita para especificar los modos de paso de parámetros.

- c. Explique cómo maneja Ada los tipos de parámetros in-out de acuerdo al tipo de dato

Primitivos: valor-resultado

No primitivos y compuestos: referencia

Ejercicio 4: Sea el siguiente programa escrito en Pascal-like

<pre>Procedure Main; var j, m, i: integer; Procedure Recibe (x:integer; y:integer); begin m:= m + 1 + y; x:=i + x + j; y:=m - 1; write (x, y, i, j, m); end;</pre>	<pre>Procedure Dos; var m:integer; begin m:= 5; Recibe(i, j); write (i, j, m); end; begin m:= 2; i:=1; j:=3; Dos; write (i, j, m); end.</pre>
---	---

- a. Arme el árbol de anidamiento sintáctico y el registro de activación de cada una de las unidades.
- b. Decir qué imprime el programa suponiendo que para todas las variables que se pasan el pasaje de parámetros es por: (Deberá hacer la pila estática y dinámica para cada caso)

i. Referencia

Current		Current	
Free		Free	
*1 RA Main		*1 RA Main	
PR		PR	
$j = 5$	$(5, 5, 6)$	$j = 8$	$(5, 8, 2)$
$m = 6$		$m = 2$	
$i = 5$		$i = 5$	
Procedure Recibe		Procedure Recibe	
Procedure Dos		Procedure Dos	
VR		VR	
*2 RA Dos		*2 RA Dos	
PR		PR	
EE(*1)		EE(*1)	
ED(*1)	$(5, 5, 5)$	ED(*1)	$(5, 8, 9)$
$m = 5$		$m = 9$	
VR		VR	
*3 RA Recibe		*3 RA Recibe	
PR		PR	
EE(*1)		EE(*1)	
ED(*2)	$(5, 5, 5, 6)$	ED(*2)	$(5, 8, 5, 2)$
$x =$		$x =$	
$y =$		$y =$	
Referencia - Estática		Referencia - Dinámica	

ii. Valor

Current		Current	
Free		Free	
*1 RA Main		*1 RA Main	
PR		PR	
$j = 3$	$(1, 3, 6)$	$j = 3$	$(1, 3, 2)$
$m = 6$		$m = 2$	
$i = 1$		$i = 1$	
Procedure Recibe		Procedure Recibe	
Procedure Dos		Procedure Dos	
VR		VR	
*2 RA Dos		*2 RA Dos	
PR		PR	
EE(*1)		EE(*1)	
ED(*1)	$(1, 3, 5)$	ED(*1)	$(1, 3, 9)$
$m = 5$		$m = 9$	
VR		VR	
*3 RA Recibe		*3 RA Recibe	
PR		PR	
EE(*1)		EE(*1)	
ED(*2)	$(5, 5, 3, 6)$	ED(*2)	$(5, 8, 3, 9)$
$x = 5$		$x = 5$	
$y = 5$		$y = 8$	
VR		VR	
Valor - Estática		Valor - Dinámica	

iii. Valor-Resultado

Current		Current	
Free		Free	
*1 RA Main		*1 RA Main	
PR		PR	
j = 35	(5,5,6)	j = 38	(5,8,2)
m = 26		m = 2	
i = 75		i = 75	
Procedure Recibe		Procedure Recibe	
Procedure Dos		Procedure Dos	
VR		VR	
*2 RA Dos		*2 RA Dos	
PR		PR	
EE(*1)		EE(*1)	
ED(*1)	(5,5,5)	ED(*1)	
m = 5		m = 89	(5,8,9)
VR		VR	
*3 RA Recibe		*3 RA Recibe	
PR		PR	
EE(*1)		EE(*1)	
ED(*2)	(5,5,5,6)	ED(*2)	(5,8,8,9)
x = 75		x = 75	
y = 35		y = 38	
VR		VR	
Valor-Resultado Estático		Dinámico	

iv. Nombre

El procedimiento es el mismo que por referencia.

v. Resultado.

Dependiendo del lenguaje se va a poder resolver o no, debido a que las variables 'x' e 'y' en el Procedure Recibe nunca son inicializadas.

- c. ¿Existió algún caso que no pudo realizarlo porque saltó algún tipo de error? Diga cuál y por qué.

En el caso de pasaje de parámetros por Resultado dependiendo del lenguaje se va a poder resolver o no, debido a que las variables 'x' e 'y' en el Procedure Recibe nunca son inicializadas.

- d. ¿Daré el mismo resultado si se trata de un lenguaje que sigue la cadena dinámica? Justifique la respuesta realizando las pilas de activación

No da el mismo resultado, esto se puede ver ejemplificado en todos los incisos de las pilas. En estas se ve la diferencia de impresión del programa.

Ejercicio 5: Suponiendo que se está ejecutando un programa con el siguiente registro de activación en memoria y se llama al procedimiento rutina(iter,vec,a). Determine el tipo de parámetro que se deben utilizar en el llamado para que los resultados sean los siguientes:

- a) (4,6,7),(4,6,7), 2, 2
- b) (3,5,6),(4,6,7), 2, 2
- c) (3,5,6),(5,5,6), 0, -1

PR
LD
LE
Iter: true
Vec:[3,5,6]
a: -1
Rutina()
VR

.....

procedura rutina(tipoParam iteracion,tipoParam vector,tipoParam vit):

```

while iteracion begin
    vit = a+1
    vector[vit] = vector[vit]+1
    iteracion = (vector[vit] mod 2)==0
end
print vec
print vector
print vit
print a

```

.....

rutina(iter,vec,a)

- a) procedure rutina(cualquiera iteracion, referencia vector, referencia/nombre vit):
- b) procedure rutina(cualquiera iteracion, valor/valor-resultado vector, referencia/nombre vit):
- c) procedure rutina(cualquiera iteracion, valor/valor-resultado vector, valor vit):

Ejercicio 6: Indique con un ejemplo el comportamiento del parámetro por nombre (en el parámetro formal) para los siguientes casos de parámetros reales:

- Un valor entero: Se comporta como el pasaje por referencia.
- Una constante: Se comporta como el pasaje por valor.
- Un elemento de un arreglo: Puede cambiar de elemento del arreglo, si se utilizó una variable para tomar el elemento pasado como parámetro, si no, se comporta como el pasaje por referencia.
- Una expresión: Se evalúa cada vez.

Ejercicio 7: Realice la pila de ejecución del siguiente programa: **a)** siguiendo la cadena estática **b)** siguiendo la cadena dinámica

```
Procedure Uno;
y, z: integer;
r1:array[1..6] of integer;
r2:array[1..5] of integer;
  Procedure Dos( nombre x, t:integer; var
io:integer; valor-resultado y:integer);
    Procedure Dos( nombre t1:integer);
      Procedure Tres;
        begin
          y:= y + 1;
          z:= z + 1;
        end;
      begin
        t1:= t1 + 1;
        t:= t + 1;
        Tres;
        t1:= t1 + 2;
        t:= t + 2;
      end;
    end;
```

```
begin
  x:= x + 1;
  t:= t + 1;
  io:= io + 1;
  x:= x + 2;
  if z =2 then Dos ( t );
end;
begin
  for y:= 1 to 6 do r1(y):= 2;
  for y:= 1 to 5 do r2(y):= 1;
  z:= 2;
  y:= 1;
  Dos( r1( y + r2( y )), r2( z ), y, z);
  for y:= 1 to 6 do write (r1(y));
  for y:= 1 to 5 do write (r2(y));
end.
```

Papel de fibra de caña de azúcar.

Current		Procedure		Tres	
F_{VR}		VR		VR	
*1 RA Uno		*4 RA Tres			
PR		PR			
$y = 7.6 \ 4.5 \ 7.2 \ 7.6 \ 7.5$		EE(*3)			
$z = 2 \ 3 \ 3$		ED(*3)			
$r1(1) = 2$		VR			
$r1(2) = 2 \ 3$					
$r1(3) = 2$					
$r1(4) = 2 \ 4$					
$r1(5) = 2$					
$r2(6) = 2$					
$r2(7) = 1$					
$r2(2) = 2 \ 3 \ 4$					
$r2(3) = 2 \ 3 \ 5$					
$r2(4) = 1$					
$r2(5) = 1$					
Procedure Dos					
VR					
*2 RA Dos					
PR					
EE(*1)					
ED(*1)					
$x = (\uparrow r1(y+r2(y)))$					
$t = (\uparrow r2(z))$					
$io =$					
$z \ y = 2 \ 3$					
Procedure Dos					
VR					
*3 RA Dos					
PR					
EE(*2)					
ED(*2)					
$t1 = (\uparrow t)$					

Estática y Dinámica
son iguales

Ejercicio 8:

- a. Indique las diferencias entre los pasajes de subprogramas como parámetros deep y shallow.

Deep: el ambiente de referencia es la unidad dónde está declarado el subprograma usado como parámetro real. Sería donde se encuentra la declaración.

Shallow: el ambiente de referencia es la unidad donde se encuentra declarado el parámetro formal. Sería donde se está recibiendo.

- b. Realice la pila estática y dinámica tanto con el pasaje de parámetros deep y shallow para el siguiente código.

<pre>Program A Var x:integer; Var y: char; Procedure B; Var h:integer; Begin h:=1+x; Write (y); C(D); Write (y); End; Procedure C (Subrutina S); Var x:integer; Var y: char; Begin x:=3; y:= "b"; x:=S(x,y) y:= "j"; Write (x,y); End;</pre>	<pre>Function D (j:integer, k:char); Begin j:=j+x; k:=y; Write (k); Return j; End; BEGIN x:=0; y:="a"; B(); Write (x,y); END.</pre>
--	--

Deep estática

Shallow estática y dinámica

Deep dinámica

8) Current	Current	Current
Free	Free	Free
*1 RA A	*1 RA A	*1 RA A
PR	PR	PR
x = 0	x = 0	x = 0
y = 'a'	y = 'a'	y = 'a'
Procedure B	Procedure B	Procedure B
Procedure C	Procedure C	Procedure C
Function D	Function D	Function D
VR	VR	VR
*2 RA B	*2 RA B	*2 RA B
PR	PR	PR
EE(*1)	EE(*1)	EE(*1)
ED(*1)	ED(*1)	ED(*1)
h = 1	h = 1	h = 1
VR	VR	VR
*3 RA C	*3 RA C	*3 RA C
PR	PR	PR
EE(*1)	EE(*1)	EE(*1)
ED(*1)	ED(*1)	ED(*1)
x = 3.6	x = 3.6	x = 3.6
y = 'b' 'j'	y = 'b' 'j'	y = 'b' 'j'
S	S	S
VR 3	VR 6	VR 6
*4 RA D	*4 RA S	*4 RA D
PR	PR	PR
EE(*1)	EE(*3)	EE(*1)
ED(*3)	ED(*3)	ED(*3)
j = 3.6	j = 3.6	j = 3.6
k = 'b' 'a'	k = 'b' 'b'	k = 'b' 'b'
VR	VR	VR

Ejercicio 9: Sea el siguiente código escrito en Pascal like

<pre>Procedure main a: array(1..5) of integer; x: integer; i;integer; Procedure Uno (tipo_pasaje m:integer) Begin x:=0; x:=x+1; m:=m+x + a(3); x:=x*2; a(3):=a(3) - 1; m:=m+1; End;</pre>	<pre>Begin For i:=1 to 5 a(i):=1; x:=3; Uno(a(x)); For i:=1 to 5 write (a(i)); End.</pre>
---	---

- Plantee diferencias, relacionada con la forma de implementación de cada uno y los resultados sobre este ejemplo considerando los siguientes tipos de pasajes parámetros nombre, referencia y valor resultado.
- ¿Qué sucede si en Uno se agrega la siguiente declaración: x: integer? Indique el resultado para cada uno de los tipos de pasajes de parámetros (nombre, referencia y valor resultado)

Ejercicio 10: Sea el siguiente un programa escrito en Pascal:

<pre>Program Uno; var x:integer; Function Dos:integer; begin x:= x + 1; return (x); end;</pre>	<pre>Procedure Tres (pasaje x:integer); begin x:= x + 5; x:= Dos + 10; end; begin x:= 8; Tres(x); write (x); end.</pre>
--	---

- a. Explique cómo simularía en Pascal el pasaje por valor-resultado y hágalo sobre este ejemplo.

Nota: No se pueden agregar más variables, ni cambiar el nombre de las que están.

<pre>Program Uno; var x:integer; Function Dos:integer; begin Dos := x + 1; end;</pre>	<pre>Function Tres (x:integer); begin x:= x + 5; Tres := Dos + 10; end; begin x:= 8; x := Tres(x); write (x); end.</pre>
---	--

- b. Transcriba este ejemplo en Ada de manera tal que el resultado de la ejecución sea diferente si el pasaje de parámetros es por referencia y luego por valor – resultado